

Sistema Mérito

Moeda Estudantil

Arquitetura Completa do Sistema

REACT + VITE · SPRING BOOT 3.3.4 · SQLITE 3

Os Quatro Atores do Ecossistema

O sistema conecta quatro atores principais:



1. **Instituições de Ensino** — Definem políticas de reconhecimento e gerenciam o programa
2. **Professores** — Reconhecem desempenho e comportamento dos alunos com moedas
3. **Alunos** — Acumulam moedas e as trocam por benefícios
4. **Empresas Parceiras** — Oferecem vantagens e descontos em troca de moedas

Este diagrama ilustra a interconexão fundamental entre as Instituições de Ensino, Professores, Alunos e Empresas Parceiras, destacando o fluxo e a colaboração no ecossistema.

Visão Geral da Arquitetura



Frontend

React + Vite

localhost:5173

Interface SPA responsiva com dashboards para Alunos e Empresas



Backend

Spring Boot 3.3.4

localhost:8080/api

API RESTful com segurança JWT e lógica de negócio transacional



Database

SQLite 3

merito.db

Banco de dados local com suporte a transações ACID e índices otimizados



Comunicação via **HTTP/JSON** com autenticação **JWT** em todas as rotas protegidas

Frontend — React + Vite

Dashboard Aluno

- Lista de Alunos
- Criar Aluno
- Deletar Aluno
- Ver Empresas

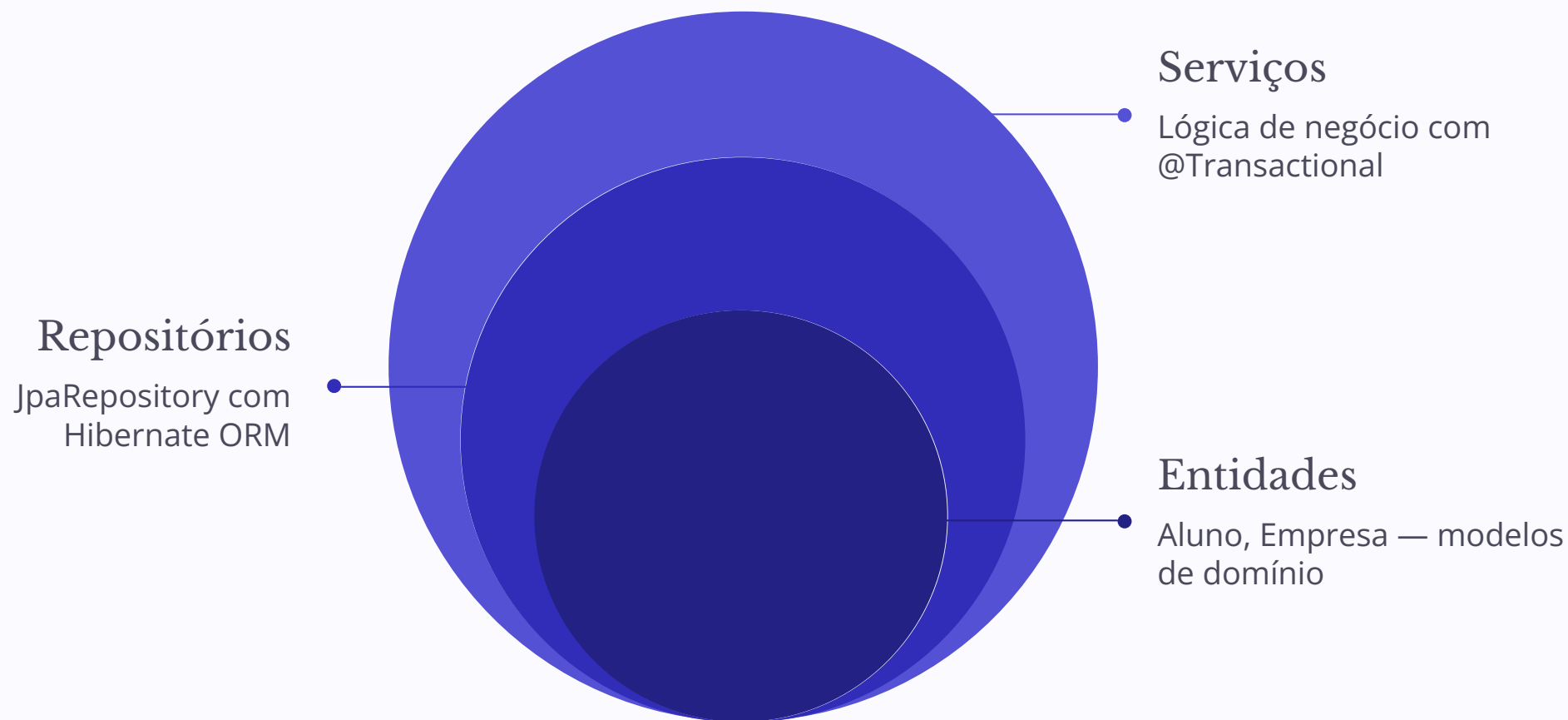
Dashboard Empresa

- Lista de Empresas
- Criar Empresa
- Deletar Empresa
- Ver Alunos

📄 API Client via **Axios** com Interceptadores JWT — token injetado automaticamente em cada requisição



Backend — 4 Camadas



Arquitetura em camadas com separação clara de responsabilidades — cada camada se comunica apenas com a camada imediatamente abaixo, garantindo coesão e baixo acoplamento.

Fluxo de Autenticação JWT

01

Acesso ao Login

Usuário acessa `/api/auth/login` com email, senha e tipo (ALUNO/EMPRESA)



Payload do JWT

`user_id · email · nome`

`tipo · iat · exp`

02

Validação BCrypt

Backend valida a senha com **BCrypt 10 rounds** — hash seguro e irreversível

03

Emissão do Token

Gera JWT assinado com **HS512**, válido por **24 horas**

04

Requisições Autenticadas

Frontend armazena token no `localStorage` e envia
`Authorization: Bearer {token}`

14 Endpoints Mapeados

Alunos — 7 rotas

- `POST /api/alunos`
- `GET /api/alunos`
- `GET /api/alunos/{id}`
- `GET /api/alunos/email/{email}`
- `GET /api/alunos/instituicao/{inst}`
- `PUT /api/alunos/{id}`
- `DELETE /api/alunos/{id}`

Empresas — 6 rotas

- `POST /api/empresas`
- `GET /api/empresas`
- `GET /api/empresas/{id}`
- `GET /api/empresas/email/{email}`
- `PUT /api/empresas/{id}`
- `DELETE /api/empresas/{id}`

Autenticação — 1 rota

- `POST /api/auth/login`

Design Patterns Utilizados

MVC

Model: Entidades JPA

View: Componentes React

Controller: REST Controllers

Layered Architecture

Presentation → Service → Data → Domain

DTO Pattern

Request: CriarAluno, AtualizarAluno, Login

Response: AlunoDTO, EmpresaDTO, LoginResponse

Transactional Pattern

@Transactional → ACID automático

@Transactional(readOnly) → Otimizado para leitura

Database — SQLite 3

Tabela: aluno

Campo	Tipo / Restrição
id	VARCHAR(36) PK
nome, email	UNIQUE
senha_hash	BCrypt
cpf, rg	CPF UNIQUE
endereco, curso	TEXT
instituicao, saldo	TEXT / DECIMAL

Tabela: empresa

Campo	Tipo / Restrição
id	VARCHAR(36) PK
nome, email	UNIQUE
senha_hash	BCrypt
cnpj	UNIQUE
descricao	TEXT
telefone	VARCHAR

✔ Índices em **email** e **CPF/CNPJ** para consultas otimizadas · Transações **ACID** garantidas

OBRIGADO!